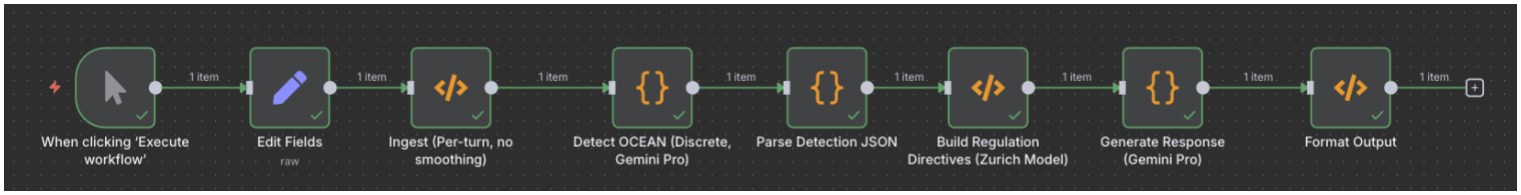
Executive Summary (MVP)

This proposal formalizes the system from *N8N Architecture v1* into buildable specifications: pipeline behavior, N8N workflow design, prompts, algorithms, state/data schemas, APIs, and validation targets. The system operates as **dialog-only, prompt-only** with **OCEAN vector** as the sole persistent memory state.

The specifications provide implementation-ready details for developers to build the personality-adaptive conversational AI system, ensuring deterministic behavior, comprehensive audit trails, and healthcare compliance requirements are met through precise technical definitions.



2.1 Current MVP (Discrete Workflow implemented)

The current prototype implements a **minimal, per-turn workflow** without memory or verification. Each conversation turn is processed independently through the following stages:

1. **Ingest** – User input is normalized into a simple context string.
2. **Detect** – An LLM infers discrete OCEAN personality traits (values of -1, 0, or 1).
3. **Parse** – The system extracts the personality vector, applying thresholds if continuous scores are returned.
4. **Regulate** – Traits are mapped to up to five communication directives using the Zurich Model.
5. **Generate** – Guided by these directives, the LLM produces a grounded reply (70–150 words, ≤2 questions).
6. **Format Output** – Results are returned as a compact JSON object with session ID, user message, and personality vector.
7. Table 1. End-to-End Turn (MVP)

Ingest	{session_id, message}	conversation_context	Normalize minimal turn context.
Detect	conversation_context	ocean_disc ∈ {−1,0,1}^5	Return JSON only; per-turn signals.
Parse	raw LLM JSON	clean ocean_disc	Strict schema; safe neutral fallback.
Regulate	ocean_disc	≤5 directives	Map traits to tone, pacing, structure.
Generate	directives, turn_text	70–150 word reply	Grounded; ≤2 questions; policy-constrained.
Format	above	{session_id, reply, ocean_disc}	Compact, auditable payload.

This MVP runs entirely in-memory: there is **no database persistence, smoothing, or refinement**. Both detection and generation calls are limited to **20 seconds**, with low creativity (temperature 0.1) for detection and higher creativity (0.7) for generation.

Performance targets mirror the architectural design: **95% of replies should return in under 2.5 seconds**, or 5.0 seconds when refinement is introduced. The emphasis is on **stable, predictable, and therapeutically consistent behavior**, rather than rapid personality shifts.

3. End-to-End Pipeline (per turn)

There are two variants:

- Target full loop (future): **ingest → detect → smooth → regulate → generate → verify → checkpoint** - Current MVP (implemented): **ingest → detect(discrete) → parse → regulate → generate → format-output**

In the MVP, the ingest step normalizes user input and creates a per-turn `conversation_context`. Detection performs LLM-based OCEAN personality inference in discrete form. The parse step extracts JSON and thresholds continuous outputs if necessary. Regulation maps discrete traits to communication directives. Generation crafts dialog-grounded replies following directives with length and question caps. The output step formats `{session_id, reply, turn_text, ocean_disc}`.

4. N8N Workflow Specification

4.0 MVP Discrete Workflow (implemented)

Source: `preliminary-studies/w9-Technical-Specifications/MVP/workflows/Discrete_workflow.json`

Nodes and contracts:

Manual Trigger

This node initiates the workflow execution via user interface interaction, serving as an entry point in n8n automation pipelines. In research terms, it functions as a manual stimulus trigger, analogous to experimental onset in behavioral studies, ensuring controlled activation of downstream processes without automated scheduling. Outputs a blank item to propagate flow, enabling testing of personality-adaptive response generation in a simulated conversational setup.

Edit Fields

Acting as a data injector, this n8n Set node configures raw input JSON with session_id and message fields for testing purposes. Succinctly, it emulates user input preprocessing in human-AI interaction paradigms, providing a structured payload that mimics real-time query ingestion. Outputs a JSON object used by subsequent nodes, facilitating isolated evaluation of discrete personality detection without live data streams.

Ingest (Per-turn, no smoothing)

This Function node normalizes incoming message data by trimming and formatting it into a per-turn conversation context prefixed as "user: <msg>". In research lingo, it performs lightweight text sanitization akin to utterance segmentation in dialogue systems, avoiding historical smoothing to focus on isolated turn analysis. Outputs session_id, clean_msg, and conversation_context, preparing atomic inputs for Big Five personality inference in adaptive AI frameworks.</msg>

Detect OCEAN (Discrete, Gemini Pro)

Utilizing n8n Code for HTTP API calls, this node queries a large language model (Gemini-1.5-Pro) to infer discrete Big Five traits (OCEAN: Openness, Conscientiousness, Extraversion, Agreeableness, Neuroticism) from turn text, yielding values in {-1, 0, 1}. It embodies trait detection in personality psychology via LLM prompting, with low temperature (0.1) for deterministic outputs and fallback to neutral zeros on errors. Outputs JSON response, enabling downstream discretization in emotion-regulation workflows.

Parse Detection JSON

This Code node extracts and validates the LLM's JSON output, parsing ocean_disc directly or thresholding continuous ocean scores (τ=0.2) to discrete equivalents if needed. In succinct terms, it applies post-hoc parsing and binarization, mirroring data cleaning in psychometric analysis to ensure robust trait vectors. Outputs refined ocean_disc, session_id, and turn_text, bridging detection to behavioral adaptation in personality-tailored interventions.

Build Regulation Directives (Zurich Model)

As a Function node, it maps discrete OCEAN values to targeted behavioral directives based on the Zurich emotion-regulation model, e.g., high Neuroticism prompts anxiety acknowledgment. This operationalizes trait-to-strategy translation in applied psychology, capping directives at five for concise guidance. Outputs directives array alongside session_id, turn_text, and ocean_disc, scaffolding adaptive response generation in supportive AI systems.

Generate Response (Gemini Pro)

This Code node leverages Gemini-1.5-Pro via API to craft a reply adhering to directives, grounded solely in user text with constraints on length (70-150 words) and questions ($\leq$2). It represents regulated output synthesis in human-centered AI, using moderate temperature (0.7) for creative yet directive-compliant responses, with error fallbacks preserving metadata. Outputs merged API data including session_id, turn_text, ocean_disc, and directives for traceable adaptive dialogues.

Format Output

Functioning as a final assembler, this Function node extracts the core reply from upstream data and compiles a compact JSON payload. In research parlance, it performs output distillation akin to result aggregation in experimental pipelines, ensuring clean, metadata-enriched deliverables. Outputs {session_id, reply, turn_text, ocean_disc}, facilitating endpoint integration in MVP prototypes for personality-aware conversational agents.

Connections (Sequential)

The workflow chains nodes linearly: Manual Trigger activates Edit Fields for input setup, flowing to Ingest for normalization, then Detect for trait inference, Parse for validation, Build Regulation for directive mapping, Generate for response creation, and Format Output for final packaging. This sequential topology mirrors a staged processing pipeline in computational psychology, enabling end-to-end evaluation of discrete OCEAN-based adaptation from input to tailored output.

4.1 Workflow Nodes — responsibilities & contracts (full workflow — planned)

Webhook Trigger Node

This n8n Webhook Trigger node serves as the API entry point, capturing HTTP POST requests containing user session data and messages to kickstart the personality-adaptive dialogue process. In research terms, it acts as a real-time stimulus receptor in interactive AI systems, ensuring seamless integration with client applications. Inputs include JSON payload with session_id (UUID) and message (text); outputs propagate these to downstream nodes, enabling controlled execution in conversational psychology experiments.

Ingest Function Node

Functioning as a data preprocessor, this n8n Function node sanitizes and structures incoming user input while maintaining a rolling history of prior exchanges. Succinctly, it performs utterance normalization akin to preprocessing in natural language understanding pipelines, extracting salient quotes for context enrichment. Inputs comprise raw_message and database-fetched history; outputs deliver clean_msg and trimmed history (limited to N turns), facilitating efficient trait inference in adaptive behavioral models.

Detect HTTP Node

This HTTP Request node invokes an LLM API to derive per-turn Big Five (OCEAN) personality traits solely from dialogue context, embodying trait detection in computational psychometrics. It operationalizes inference via structured prompting, yielding raw_ocean scores, trait_conf confidences, and evidence_quotes. Inputs are clean_msg and history; outputs provide these metrics, with configuration for JSON-formatted responses from endpoints like OpenAI, ensuring deterministic analysis in personality dynamics studies.

Smooth Function Node

As an n8n Function node, it applies exponential moving average (EMA) smoothing with confidence weighting to stabilize personality estimates across turns, incorporating delta caps for gradual updates. In research lingo, this mirrors temporal filtering in longitudinal trait modeling, computing a stable flag based on threshold criteria. Inputs include raw_ocean, ocean_prev, and trait_conf; outputs are refined ocean vector and stable boolean, supporting robust state tracking in interactive AI therapies.

Regulate Function Node

This Function node translates smoothed OCEAN traits into behavioral directives via linear mappings, resolving conflicts to form a concise policy_plan (up to 4 items). It represents regulatory adaptation in personality psychology, tailoring interaction styles to user traits. Inputs are ocean and stable; outputs deliver policy_plan, enabling downstream generation of responses aligned with evidence-based emotion regulation frameworks like the Zurich model.

Generate HTTP Node

Utilizing an HTTP Request to an LLM, this node synthesizes a draft_reply adhering to the policy_plan while remaining strictly grounded in dialogue history. Succinctly, it enacts controlled generation in human-AI interaction research, enforcing bounds on external knowledge introduction. Inputs comprise policy_plan and history; outputs yield draft_reply, configured with prompts emphasizing therapeutic helpfulness and conversational fidelity.

Verify Function Node

This n8n Function node audits the draft_reply for compliance, detecting deviations in policy adherence, novel claims, or grounding lapses through heuristic checks. In succinct terms, it performs post-generation validation akin to fidelity assessments in behavioral interventions, outputting a decision (accept/refine) and reasons. Inputs include draft_reply, policy_plan, and history; outputs guide conditional refinement, ensuring ethical and trait-aligned outputs.

Refine HTTP Node (conditional)

Activated on verification failure, this HTTP Request node refines the draft via targeted LLM calls, making minimal adjustments without adding content. It embodies iterative correction in adaptive systems research, focusing on tone tweaks and claim removals. Inputs are draft_reply, reasons, policy_plan, and history; outputs produce final_reply, with prompts designed for precision fixes in personality-tailored dialogues.

Deliver Function Node

As a final Function node, it assembles and formats the API response, including the reply and metadata previews for client consumption. Research-wise, it acts as an output aggregator in experimental pipelines, providing transparency into system state. Inputs encompass final_reply (or draft), policy_plan, and ocean; outputs form a JSON payload, facilitating seamless integration in real-world conversational AI deployments.

Database Query Node (initial)

This Postgres node fetches prior session data at workflow onset, retrieving history and ocean_prev for continuity. It operationalizes state retrieval in persistent dialogue systems, akin to memory recall in cognitive models. Inputs are session_id from the webhook; outputs include history, ocean_prev, and metadata, configured via SELECT queries on turns and state_snapshots tables.

Database Write Node (final)

Positioned at workflow end, this Postgres node persists updated state, logging turns and snapshots for auditability and future retrieval. In research parlance, it ensures data integrity in longitudinal studies, storing immutable records with execution IDs. Inputs comprise updated history, ocean, policy_plan, flags, and timings; outputs confirm database insertion via INSERT on turns and state_snapshots, supporting scalable session management.

4.2 Workflow Connections & routing (full workflow — differ)

Sequential workflow connections: 1. `Webhook Trigger → Database Query Node` (retrieve session history) 2. `Database Query → Ingest Function → Detect HTTP → Smooth Function → Regulate Function → Generate HTTP → Verify Function` form the main processing pipeline with deterministic sequential execution.

Conditional routing: - `Verify Function → IF Node` evaluates decision field - IF `decision == "refine"` → `Refine HTTP Node → Deliver Function` - IF `decision == "accept"` → `Deliver Function` - `Deliver Function → Database Write Node` for final state persistence

Error handling connections: - HTTP Request nodes connect to `Error Handling Function` on failure - Error handler routes to neutral response delivery while preserving conversation state - All execution paths lead to Database Write for audit trail maintenance

The workflow maintains deterministic and replayable execution with explicit failure handling through N8N's built-in error routing and retry mechanisms.

5. Core Algorithms

5.1 Algorithm 1 Detecting OCEAN Traits

Inputs: `clean_msg` (single turn text). Output: `ocean_disc` with values in {-1, 0, 1} for {O, C, E, A, N}.

- Build prompt with strict JSON-only instruction returning `{"ocean_disc":{...}}` for THIS TURN ONLY. - Call LLM with temperature 0.1, max_tokens 200, 20s timeout. - Parse response: - Attempt `JSON.parse(content)`; if fails, extract first `{...}` span and parse. - If only continuous `ocean` is present, convert to discrete using threshold $\tau = 0.2$: - `disc(v) = 1 if v ≥ τ; -1 if v ≤ -τ; else 0` applied per trait. - Validate keys and values; on any invalid/missing field, set neutral `{O:0,C:0,E:0,A:0,N:0}`. - Return `ocean_disc`.

```
Input: task description T_task, datasets D, score function h, number of retrieved models M,
  {T_model_i, T_code_i} = A_retriever(T_task)
for i = 1 to M do
    s_init = A_init(T_task, T_model_i, T_code_i)
    Evaluate h(s_init) using D
    end for
```

```
s_0 ← s(1)
h_best ← h(s_0)
for i = 2 to M do
    s_candidate ← A_merger(s_0, s_init(i))
    Evaluate h(s_candidate) using D
    if h(s_candidate) ≥ h_best then
        s_0 ← s_candidate
        h_best ← h(s_0)
    else
        break
    end if
end for
Output: initial solution s_0
```

5.2 Algorithm 2 Building Regulation Directives

Inputs: `directives` (≤5 strings), `turn_text` (user text). Output: `reply`.

- Construct system content: “Follow these behavior directives strictly: [directives]. Constraints: grounded in user text only; ≤2 questions; 70–150 words.” - Provide `turn_text` as user content (fallback to a safe generic instruction if empty). - Call LLM with temperature 0.7, max_tokens 220, 20s timeout. - Post-processing: trim; no verification/refinement pass in MVP (constraints enforced via prompt only). - Error fallback: short supportive message with breathing tip; include `session_id`, `ocean_disc`, and `directives` in the JSON payload for traceability.

Table 2. OCEAN → Directive Mapping (illustrative)

O +1 / −1	Invite small exploration / Prefer familiar	"Suggest one new approach" / "Use familiar examples"
C +1 / −1	Structure steps / Keep flexible	"Provide 2–3 concise steps" / "Offer optional paths"
E +1 / −1	Energetic tone / Calm tone	"Use active verbs" / "Use reflective phrasing"

A +1 / −1	Warm, collaborative / Neutral	"Acknowledge briefly" / "State points succinctly"
N +1 / −1	Reassure more / Reassure less	"Normalize concerns in one line" / "Avoid over-reassurance"

```
Input: task description T_task, datasets D, score function h, number of retrieved models M,
  {T_model_i, T_code_i} = A_retriever(T_task)
for i = 1 to M do
    s_init = A_init(T_task, T_model_i, T_code_i)
    Evaluate h(s_init) using D
    end for
    s_0 ← s(1)
    h_best ← h(s_0)
    for i = 2 to M do
        s_candidate ← A_merger(s_0, s_init(i))
        Evaluate h(s_candidate) using D
        if h(s_candidate) ≥ h_best then
            s_0 ← s_candidate
            h_best ← h(s_0)
        else
            break
        end if
    end for
Output: initial solution s_0
```

6. Prompt Interfaces (skeletons — MVP emphasis)

Detection Prompt (returns JSON)

Structured output requiring exact JSON keys with no prose commentary:

```
{"ocean":{"O":0.2,"C":-0.1,"E":0.0,"A":0.6,"N":-0.4},
 "trait_conf":{"O":0.7,"C":0.6,"E":0.55,"A":0.75,"N":0.7},
 "evidence_quotes":["User said 'I prefer trying new approaches'","Shows structured
thinking patterns"]}
```

The prompt analyzes conversation context to infer OCEAN traits with confidence scores and supporting evidence quotes from the dialog.

Detection Prompt (MVP discrete)

Return JSON only with signed discrete traits for this turn only. No prose commentary:

```
{"ocean_disc":{"O":-1|0|1,"C":-1|0|1,"E":-1|0|1,"A":-1|0|1,"N":-1|0|1}}
```

Regulation Prompt (policy plan — MVP uses ocean_disc only)

Input consists of discrete `ocean_disc` (MVP) or smoothed `ocean` (future) plus brief conversation history. Output generates ≤4 behavioral directives targeting specific style dimensions:

Example output: `["validate user concerns briefly","ask one open-ended question","use concise sentences","avoid imperative language"]`

Generation Prompt (quote-and-bound)

Instructions emphasize using only dialog content while adhering to policy directives. Response length and question limits are enforced through prompt constraints. All assertions must be grounded in conversation history.

Generation Prompt (MVP constraints)

Follow these behavior directives strictly; stay grounded in user text only; 70–150 words; ask at most 1–2 questions. Example system content construction:

```
"You are a supportive assistant. Follow these behavior directives strictly:
[<directives JSON>]. Constraints: stay grounded in the user text only; 1-2 questions
max; 70-150 words."
```

Verify / Refine Prompts

Verifier returns structured assessment: `{pass: bool, reasons: ["novel claim detected", "policy directive violated"]}`. Refine prompt modifies tone and removes novel claims while preserving core response content and dialog grounding.

7. Authoritative State & Data Schemas

7.1 State JSON (single source of truth)

The system persists this complete state object after every turn, providing the single authoritative source for session state and enabling deterministic replay:

```
{
```

```
  "session_id":"uuid",
  "turn_index":17,
  "history":[{"role":"user","text":"How are you feeling today?"},{"role":"assistant","text":"I'm doing well, thank you for asking. How has your day been going?"}],
  "ocean":{"O":0.4,"C":-0.1,"E":0.0,"A":0.5,"N":-0.3},
  "trait_conf":{"O":0.72,"C":0.61,"E":0.58,"A":0.74,"N":0.69},
  "stable":true,
  "policy_plan":["validate briefly","ask one open question","concise sentences","avoid imperatives"],
  "flags":{"refined_once":false,"neutral_fallback":false},
  "timings_ms":{"detect":380,"generate":920,"verify":210,"total":1510},
  "last_updated":"2024-09-02T14:33:00Z"
}
```


This unified object simplifies replay scenarios, system analysis, and compliance auditing by maintaining complete conversation state with personality estimates, behavioral policies, and execution metadata in a single retrievable record.

7.2 Database schema (research-minimal)

PostgreSQL schema optimized for personality-adaptive conversation management:

```
-- Core session management
CREATE TABLE sessions (
    session_id UUID PRIMARY KEY,
    created_at TIMESTAMP DEFAULT NOW(),
    last_updated TIMESTAMP DEFAULT NOW()
);

-- Individual conversation turns
CREATE TABLE turns (
    id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES sessions(session_id),
    turn_index INTEGER NOT NULL,
    role VARCHAR(10) NOT NULL, -- 'user' or 'assistant'
    text TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Authoritative state snapshots
CREATE TABLE state_snapshots (
    id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES sessions(session_id),
    turn_index INTEGER NOT NULL,
    state_json JSONB NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Performance indexes
CREATE INDEX idx_snapshots_session_turn ON state_snapshots(session_id, turn_index);
CREATE INDEX idx_turns_session_created ON turns(session_id, created_at);
CREATE INDEX idx_sessions_updated ON sessions(last_updated);
```

The JSONB storage enables efficient querying of personality states and policy configurations while maintaining schema flexibility for research iterations.

8. API Specification (N8N Webhooks)

Core Endpoints

POST /webhook/chat *(N8N Webhook Trigger)* - Request: `{session_id: "uuid", message: "user input text"}` - Response: `{reply: "assistant response", policy_plan: ["directive1", "directive2"], ocean_preview: {"O":0.4,"C":-0.1,"E":0.0,"A":0.5,"N":-0.3}, timings_ms: {"detect":380,"generate":920,"verify":210}, execution_id: "n8n_execution_uuid"}` - Implementation: N8N webhook trigger node initiating the personality detection workflow

GET /api/session/{id}/state *(FastAPI/Next.js API Route)* - Response: Complete state JSON object from latest turn - Implementation: Direct database query outside N8N workflow

GET /api/session/{id}/summary *(FastAPI/Next.js API Route)* - Response: `{summary_bullets: ["Key conversation points"], ocean: {...}, last_directives: [...], turn_count: 17}` - Implementation: Database aggregation query with personality state summary

POST /webhook/session/reset *(N8N Webhook Trigger)* - Request: `{session_id: "uuid"}` - Response: `{status: "success", message: "Session reset to neutral state"}` - Effect: Triggers N8N workflow to clear history and reset OCEAN to neutral [0,0,0,0,0]

N8N Workflow Management

- **Workflow Execution History**: Available through N8N UI for debugging and audit trails - **Webhook URLs**: Generated by N8N for each workflow trigger endpoint - **Error Handling**: N8N built-in error catching with custom error response workflows - **Monitoring**: N8N execution logs provide detailed workflow performance metrics

8.1 MVP Execution Mode & Response

- Execution Mode: Manual trigger (`When clicking 'Execute workflow'`), no webhook used - Input: Provided in `Edit Fields` node as raw JSON `{session_id, message}` - Output payload (from `Format Output`):

```
{
  "session_id": "uuid",
  "reply": "assistant response",
  "turn_text": "original user message",
  "ocean_disc": {"O": 0, "C": 0, "E": 0, "A": 0, "N": 0}
}
```

9. Performance & Reliability

The system inherits latency targets from Architecture v1: P95 2.5s standard responses, 5.0s with refinement. Timeout configuration includes 30s total request timeout, 15s per LLM call, and single refinement budget per turn.

MVP overrides (Discrete workflow): - 20s timeout per LLM call (n8n code node HTTP helper) - No smoothing, verification, or refinement in the execution path - Deterministic fallbacks: neutral `ocean_disc` on detect errors; supportive stock reply on generate errors

Neutral fallback activates on system failures, returning conversation-appropriate responses while preserving session state. Circuit breakers protect against LLM provider failures with exponential backoff retry patterns. Short context windows (salient quote extraction) and prompt caching optimize performance while maintaining conversation quality.

Bounded retry mechanisms prevent cascade failures while comprehensive error logging enables rapid debugging and system improvement in production healthcare environments.

10. Observability

Execution Tracing

Per-node timing collection in `timings_ms` object captures detect, smooth, regulate, generate, verify, and refine execution durations. Prompt and model version tags enable A/B testing and regression analysis across system updates.

MVP note: Only detect and generate incur LLM timings; parse/regulate/format are measured via node execution logs. Additional MVP guidance: Capture node-level logs for directives and final `ocean_disc` to support troubleshooting without database state.

Decision Logging

Verification decisions (accept/refine) are logged with specific failure reasons enabling system optimization and compliance monitoring. Example trace record:

```
{
  "trace_id": "uuid",
  "session_id": "uuid",
  "turn_index": 17,
  "node_timings": {"detect": 380, "generate": 920, "verify": 210},
  "decisions": {"verify_result": "refine", "refine_reason": "novel claim detected"},
  "state_checkpoint_id": "checkpoint_uuid",
  "model_versions": {"detect": "gpt-4-v1.2", "generate": "gpt-4-v1.2"}
}
```

This comprehensive logging supports debugging, performance optimization, and regulatory compliance requirements for healthcare deployments.

11. Security & Privacy (framework-agnostic)

The dialog-only grounding approach ensures no external data dependencies or privacy leakage through external API calls. PII redaction operates through pre-processing hooks before state persistence. Role-based access control (RBAC) restricts state access to authorized personnel with configurable data retention policies supporting GDPR and HIPAA compliance.

Audit logging captures all personality detection and regulation decisions with immutable timestamps. Session state encryption protects sensitive conversation data at rest while maintaining queryability for compliance and debugging purposes.

12. Testing & Evaluation

Automated Testing Framework

Scripted dialog replay enables deterministic testing with conversation scenarios and expected personality adaptations. Regression testing validates prompt version changes against established behavioral baselines.

Performance Metrics

- **Stability Time**: Turns required to achieve stable personality detection (target: ≤5 turns) - **Oscillation Rate**: Frequency of personality estimate changes after stability (target: <10%) - **Policy Adherence Score**: Compliance with behavioral directives (target: >90%) - **Grounding Violations**: Novel claims not supported by dialog (target: <5%) - **P95 Latency**: Response time distribution (target: 2.5s/5.0s with refinement)

Evaluation Harness

Minimal testing infrastructure provides conversation simulation, personality ground truth comparison, and automated metric collection enabling continuous system validation and improvement cycles.

Table 3. Evaluation Plan (targets for small study)

Stability	Oscillation after initial turns	<10%	Avoid jarring tone shifts.
Adherence	Policy adherence score	>90%	Directives reliably shape replies.
Grounding	Novel claim rate	<5%	Maintain dialog-bounded content.
Latency	P95 per turn	≤2.5s	Keep interaction responsive.

13. Deployment Notes

Container Architecture

Containerized services enable independent scaling with N8N workflow engine connecting to shared PostgreSQL database. N8N containers handle workflow execution while Next.js frontend containers serve client interfaces and supplementary API routes.

Deployment Strategies

Blue/green deployment patterns support prompt version updates with zero downtime. Canary releases enable gradual rollout of model or prompt changes with automatic rollback on performance degradation.

Configuration Management

Environment variables control model identifiers, prompt version tags, timeout values, and database connections. Configuration validation ensures consistent deployment across development, staging, and production environments.

{ "name": "Personality Adaptive Chat Workflow", "nodes": [{ "id": "webhook-trigger", "name": "Chat Webhook", "type": "n8n-nodes-base.webhook", "position": [250, 300], "parameters": { "httpMethod": "POST", "path": "chat", "responseMode": "responseNode" } }, { "id": "db-query-history", "name": "Get Session History", "type": "n8n-nodes-base.postgres", "position": [450, 300], "parameters": { "operation": "executeQuery", "query": "SELECT state_json FROM state_snapshots WHERE session_id = $1 ORDER BY turn_index DESC LIMIT 1" } }, { "id": "ingest-function", "name": "Ingest & Normalize", "type": "n8n-nodes-base.function", "position": [650, 300], "parameters": { "functionCode": "// Normalize input and update conversation history" } }, { "id": "detect-llm", "name": "Personality Detection", "type": "n8n-nodes-base.httpRequest", "position": [850, 300], "parameters": { "method": "POST", "url": "https://api.openai.com/v1/chat/completions", "authentication": "predefinedCredentialType", "nodeCredentialType": "openAiApi" } }, { "id": "smooth-function", "name": "Smooth Personality", "type": "n8n-nodes-base.function", "position": [1050, 300], "parameters": { "functionCode": "// EMA smoothing with confidence weighting" } }], "connections": { "webhook-trigger": {"main": [["db-query-history"]]}, "db-query-history": {"main": [["ingest-function"]]}, "ingest-function": {"main": [["detect-llm"]]}, "detect-llm": {"main": [["smooth-function"]]} } }

```
**N8N Workflow Features:**
- **Webhook-triggered execution** for real-time chat processing
- **Database integration** for session persistence and history retrieval
- **HTTP nodes** for LLM API calls (detection, generation, refinement)
- **Function nodes** for personality smoothing, policy regulation, and verification
- **Conditional routing** through IF nodes for refinement decisions
- **Error handling** through N8N's built-in error routing capabilities
- **Execution logging** through N8N's native workflow monitoring

**Implementation Notes:**
- Complete workflow includes additional nodes for regulate, generate, verify, refine,
deliver, and database write operations
- Function node JavaScript code implements the core algorithms from Section 5
- HTTP Request nodes handle all LLM interactions with proper authentication
- Postgres nodes manage session state persistence and retrieval
- Conditional IF nodes route workflow execution based on verification results
```

```
**Algorithm 1: Generating an initial solution**
```

1: Input: task description Ttask, datasets D, score function h, number of retrieved models M, 2: {T i model, T i code}M i=1 = Aretriever (Ttask) 3: for i = 1 to M do 4: si init = Ainit (Ttask, T i model, T i code) 5: Evaluate h(si init) using D 6: end for 7: $s0 \leftarrow s\pi(1)$ init 8: $h$best $\leftarrow h(s0)$ 9: for i = 2 to M do 10: $s$candidate ← Amerger ($s0$, $s\pi(i)$ init) 11: Evaluate h($s$candidate) using D 12: if h($s$candidate) ≥ $h$best then 13: $s0 \leftarrow s$candidate 14: $h$best $\leftarrow h(s0)$ 15: else 16: break 17: end if 18: end for 19: Output: initial solution $s0$

```
### B.2 Detection (MVP discrete) — pseudocode

**Algorithm 2: OCEAN discrete detection (per turn)**
```

1: Inputs: clean turn text m, threshold τ = 0.2, model id, endpoint URL, 2: temperature = 0.1, max_tokens = 200, timeout = 20s 3: if m is empty then 4: return ocean_disc = {O:0, C:0, E:0, A:0, N:0} 5: end if 6: Build prompt requiring JSON-only: {"ocean_disc":{"O":-1|0|1,"C":-1|0|1,"E":-1|0|1,"A":-1|0|1,"N":-1|0|1}} 7: res ← LLMCall(model, endpoint, system_prompt, user = "Text: " + m, 8: temperature, max_tokens, timeout) 9: content ← res.choices[0].message.content or "" 10: try 11: parsed ← JSON.parse(content) 12: catch 13: jspan ← first JSON object extracted from content using /\{[\s\S]\}/ *14: parsed ← (jspan ? JSON.parse(jspan) : {}) 15: end try 16: if parsed.ocean_disc exists and all values ∈ {-1,0,1} then 17: return parsed.ocean_disc 18: else if parsed.ocean exists then 19: define disc(v) = (v ≥ τ ? 1 : (v ≤ -τ ? -1 : 0)) 20: return {O: disc(parsed.ocean.O||0), C: disc(parsed.ocean.C||0), 21: E: disc(parsed.ocean.E||0), A: disc(parsed.ocean.A||0), 22: N: disc(parsed.ocean.N||0)} 23: else 24: return {O:0, C:0, E:0, A:0, N:0} 25: end if 26: (Optional) Self-consistency: repeat k times; per-trait majority vote; ties → 0*

Analyze this conversation to infer personality traits. Return JSON only: {"ocean": {"O": float, "C": float, "E": float, "A": float, "N": float}, "trait_conf": {"O": float, "C": float, "E": float, "A": float, "N": float}, "evidence_quotes": [string, string, ...]} Values in [-1,1], confidence in [0,1]. No prose commentary.

```
### Regulation Prompt
```

Given OCEAN traits and conversation context, generate ≤4 behavioral directives. Input: {"ocean": {...}, "stable": bool, "history": [...]} Output: ["directive focusing on warmth/tone", "directive on directness", ...] Target dimensions: warmth, directness/hedging, assertiveness, pacing, question ratio.

```
### Generation Prompt
```

Generate response following directives. CRITICAL: Only use conversation content. Policy directives: [...] Conversation: [...] Requirements: Quote/paraphrase dialog only, follow directives, cap questions at 1-2.

```
### Verify Prompt
```

Check response for: (1) Policy directive compliance (2) Dialog grounding Response: "..." Policy: [...] History: [...] Return: {"pass": bool, "reasons": [string, ...]} Flag novel claims not supported by conversation.

Appendix A — N8N Workflow Configuration (JSON Schema)

Appendix B — Algorithms

B.1 Algorithm for generating an initial solution

Appendix C — Prompt Skeletons

Detection Prompt